Hexagons are the Bestagons: A Novel Neural Network Topology

Daniel Lopez Cezareo Rodriguez
SimpleRose Inc.,
St. Louis, MO
{daniel, cezareo}@simplerose.com

May 2026

Abstract

We propose and establish baselines for a novel, albeit impractical neural network topology that departs from traditional Cartesian grid layouts of neurons and connections and employs a hexagonal lattice enabling neighboring networks with overlapping weights, Hexagonal Neural Networks. Inspired by extreme behavior compartmentalization behavior disorders observed in the general populace, this network structure focuses on the influence of directional training on overlapping weights. Early baselines reveals new learning behaviors through loss and accuracy iterations and consistent improvement from single direction propagation which has widespread usage in modern day commercial deep learning structures.

1 Introduction

1.1 Architectural Directionality as an Inductive Bias

Multilayer Perceptrons (MLPs) are traditionally structured as a unidirectional chain of connected layers, where each layer sequentially connects to the next. This imposes a simple ordered sequence of transformations on the input data, where each layer defines a projection from one vector space to another, along with some activation function. Hidden layers can then be interpreted as intermediate representations whose structure depends on the imposed ordering of the network.

Through these variations, many inductive biases have been successfully encoded such as convolutional locality, invariance to translation, rotation, scaling, and other geometric transformations. While many variations of this base model have been iterated on with respect to shared weights, activation functions, and other hyperparameters, the core architectural ordering of the network remains the same and only a few deviations encode geometric transformations of the data, such as 2D or 3D convolution layers popularized in computer vision tasks. Convolutional Neural Networks (CNNs) encode locality and translation equivariance through shared convolutional kernels. Extensions such as group-equivariant CNNs impose additional transformation structure, including rotation or reflection equivariance, by tying parameters across transformations.

1.2 Hexagonal Neural Networks

Here, we introduce Hexagonal Neural Networks (HexNNs) that generalizes MLPs into a rotationally symmetric, multi-directional structure.

This architecture arranges nodes in a two-dimensional hexagonal topology rather than a single ordered layer stack, like the example shown in Figure 1. This allows for the same set of nodes and edges to be interpreted and traversed under multiple directed orientations. An orientation H_i is a directed acyclic subgraph induced by choosing one of the six directed axes of the hexagonal topology. Thusly, it is now possible to train one orientation while evalu-

ating how that update affects the other orientations. This yields a central question: whether this directional weight sharing produces measurable transfer or interference between orientations.

Because each orientation exposes input and output layers of equal width, the present formulation is most natural for tasks where input and output vectors have the same dimensionality, such as reconstruction, identity mappings, or vector-to-vector regression. Classification tasks may still be represented, but may require additional post processing layers in case the number of features is not the same as the number of classes.

Given a size parameter, n, the network has equal n length input and output layers, with hidden layers symmetrically increasing and decreasing to $2n - 1$. Crucially, each node in layer i is connected to all nodes in layer $i + 1$, but unlike MLPs, this bipartite adjacency is true for all three cardinal directions of a hexagon. This allows for rotation-based reinterpretation of the same input data.

This design enables six distinct orientations of the same network, each corresponding to a 60° rotation of the hexagonal structure. We index orientations as $H_0, ..., H_5$ starting with the default orientation H_0 being the top-to-bottom orientation. The opposite orientation of H_i is $H_{(i+3) \bmod 6}$.

1.3 Open Questions

As we can see, this new change in architecture may have some interesting implications for the training of the network. What are the implications of training the network in one direction, and how does that impact the training of the network in other directions? What effects will this have on the performance of the network in other directions? Is there an optimal sequencing of training with intermitten rotations that can yield lower overall loss in any one network or a mixture of experts (MoE) approach that can yield lower overall loss than the MLP equivalent?

1.4 Contributions

In this paper, we present a memory efficient implementation using over padded state vectors and orientation specific adjacency matrices. We also establish baselines for HexNNs and their equivalent MLPs and compare the performance of the two architectures across various datasets and hyper parameter configurations.

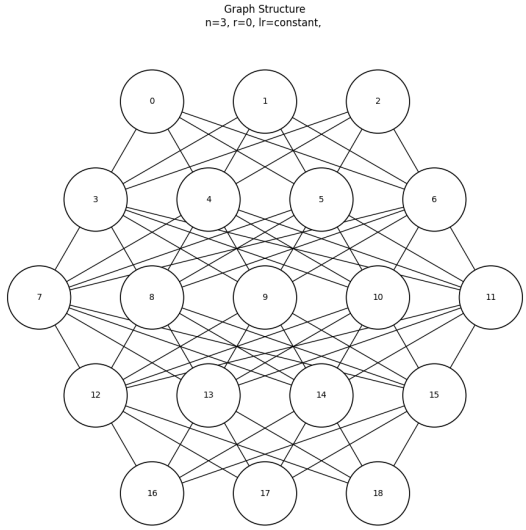

Figure 1: A Hexnet with N=3 and default rotation H_0

We also present the inductive bias: [blank] Unlike MLPs, HexNNs are able to present [blank] which can tell us if the data is [blank]. This could be useful for [blank] tasks where modern day neural networks are not able to perform well.

2 Background

2.1 MLPs and Layered Function Approximation

MLPs can be described as a composition of affine transformations and (typically) non linear activation functions. Iterations on derivative architectures modify this basic pattern with tricks like weight sharing, recurrence, attention blocks, or structured sparsity.

However, they still maintain the feedforward directionality of the network.

Moreover, standard dense MLPs are not able to encode more complex inductive biases such as rotational invariance or translational invariance by themselves. These biases must be introduced through some creative license on network features like architecture, data representation, or training procedure.

HexNNs, as introduced in this paper, keeps this adjacent-layer bipartite structure from MLPs but changes its own topology to allow for multiple feedforward interpretations from the same graph, potentially allowing for more complex inductive biases.

2.2 Symmetry and Equivariance in Neural Networks

Convolutional Neural Networks (CNNs) are a prime example of how the architural constraints of the input domain can be used to encode more complex inductive biases. They encode translational equivariance and locality of features in the input data by using convolutional filters that are "slid" across the input data. Their derivative architecture use geometric iterations on their kernels and inputs to help them unlock higher level inductive biases: Group-equivariants CNNS encode the symmetry of the input data itself to achieve rotation, reflection, or other group equivariances equivariance and Spherical CNNs extend the symmetry ideas to spherical domains for rotational equivariance. Capsule Networks project their input data into higher dimensionality with their vector interpretaion and yield complex understandings of part-whole relationships in their latent space.

All of these architectures, however, still impose a feed forward directionality on the layer ordering. In the case of HexNNs, the symmetry is imposed on the network topology itself, not only the input domain or filter group. This allows for the same node/weight structure to be reinterpreted directionally with abstraction bleeding between the different directions.

2.3 Graph Neural Networks

Graph neural networks operate on data whose structure is represented by a graph. In message-passing GNNs, node representations are updated by aggregating information from neighboring nodes according to an adjacency relation.

HexNNs also use an adjacency structure, but the present formulation differs from standard GNNs in two ways: First, the graph defines the architecture itself rather than the input data. Second, computation proceeds through orientation-specific feedforward traversals rather than repeated permutation-invariant neighborhood aggregation. For this reason, the current HexNN implementation is better understood as a graph-structured or sparse MLP topology rather than a conventional message-passing GNN.

2.4 Parameter Sharing, Ensembles, and Multi-Head Models

Parameter sharing is commonly used to reduce model size and impose structure. CNNs share kernels across spatial locations, recurrent networks reuse transition parameters across time, and multi-task models often share a representation backbone across related objectives. HexNNs use a different form of sharing: orientation-specific subnetworks are coupled through a common underlying graph. Training one orientation may therefore alter the behavior of other orientations, creating measurable transfer or interference.

HexNNs can also be compared with ensembles. Unlike an ensemble of independent models, a HexNN does not maintain separate parameters for each orientation. Its directional subnetworks are not independent; their behavior is coupled by construction.

2.5 Continual Learning and Interference

Moreover, human cognition and artificial intelligence are known to face common challenges, such as managing knowledge across diverse tasks without losing prior knowledge gain. This is known as catastrophic forgetting, where a network trained on one task is unable to perform well on a different task. This is a known challenge in artificial intelligence, and it is a known challenge in human cognition.

2.6 Cognitive Motivation

Foundational architectural models that have been proposed are able to uncover different inductive biases that are not possible with the traditional MLP architecture using natural design as motivation. CNNs, for example, discovered feature invariance after modeling the convolutional kernals after the visual cortex and its neurophysiological design. RNNs and LSTMs resemble the neurophysiological self looping design of the hippocampus and its role in memory and spatial navigation and this was able to introduce the concept of recurrent connections and the idea of memory cells in artificial intelligence. So consider the phenomenon found in people who have a dissociative identity disorder (DID), where memories and personalities are split between identities despite having the same brain. To borrow speculative language from Douglas Hofstadter, we can imagine a network toplogy that is able to hold different representations of the same data in the same hardware and each "strange loop" working as its own coherent identity but potentially affecting the learned experiences with other identities.

To be clear, this paper does not model DID or aim to benchmark DID like symptoms in a clear method like the research proposed by [blank]. DID was the human based inspiration given its similar characteristics to extremely compartmentalized behavior, something not usually found in highly emergent cognitive systems, such as humans. This inspiration also comes from the primary author's own trauma with a mother with split identities, and a father with hemiplegia after a hemoraggic stroke impaired his ability to move his left side. The cognitive analogy is strictly heuristic, not empirical.

3 Implementation

3.1 Topological Definition

A one directional view hexagonal neural network is a multilayer perceptron network whose layer sizes increase and then decrease symmetrically, forming a hexagon-like outline if it were to be drawn in 2D. As expected, all neurons in any given layer are fully connected to all neurons in the subsequent layer. However, due to the hexagonal nature of the framework, there are three distinct directions in which one could perform traditional forward and backward neural network propagation. This renders the structure to contain a unique corresponding arrangement of connections and shared weights given the direction.

These shared weights may influence the internal network's representation of data in their high-dimensional embedding from one propagation direction to another in strange ways. The shared weights of, say, the first semantic layer of H_1 will always have at least one shared weight with H_2 (the relative left-most neuron from H_2's point of view) up until the halfway point of the network. This overlap is small but non-neglible as these weights are now subject to the gradient descent paths of two distinct target local optimas. Experimentally, we actually see equivalent or better another local optimum, outperforming the two original networks had they been started with the same random seed patterns for their respective weights across the benchmark set (detailed below). Since each layer represents a layer of abstraction away from the input reality towards the target reality, it could be speculated that any one layer's abstraction has a cross abstractive coherence bias on another's direction and thus representation.

3.2 The Greek Stuff

The layer sizes of a HexNet of a parameter size n follow an explicit pattern:

$$L = [n, n+1, ...2n-1, ..., n+1, n]$$

with a $2n-1$ total layers as well. The total number of nodes in a network, however, is:

$$T = \sum_{k=0}^{2n-2} (n + \min(k, 2n-2-k))$$

3.2.1 Weight Indexing and Layer Sets

To be explicit, if we visualize the network with the layers centered and stacked vertically (i.e. with the

smallest layers at the top and bottom, widest in the middle), then we have the following directions:

- H_0**:** Top-to-bottom layers (default hexagonal structure)
- H_1**:** Top-right to bottom-left diagonal
- H_2**:** Bottom-right to top-left diagonal
- H_3**:** Bottom-to-top layers (reverse of H_0)
- H_4**:** Bottom left to top-right diagonal
- H_5**:** Top left to bottom-right diagonal

3.2.2 Global W

For mathematical simplicity, we employ a global weight matrix W, a adjacency matrix of size TxT that can represent all possible connections regardless of rotation. Given that H_0 is directionally opposite H_4, they would share exactly the same weights. While empirical trials on MLPs show that bidirectional training on the same weights show catastrophic loss, (find reference), we could theoretically use different cells to represent that as a separate network and still maintain directional-abstraction bleeding between immediately neighboring dimensions. If this were the case, then W would not be symmetric. An example of this is shown in Figure 2.

3.2.3 Directional W

We can represent the static MLP view of any arbitrary rotation, $r \in 0, 1, 2, 3, 4, 5$ as W_i with a mask over W enforcing a layered feedforward structure.

For the next few mathematical definitions, let the **Vertical Direction** orientation of the network, H_1, be the assumed direction. Furthermore, let's assume the numbering of the network to start from the top left index, traverse left to right, and then top to bottom. Let:

- $A^i \in \{0, 1\}^{T \times T}$ be the binary adjacency matrix for arbitrary direction H_i
- For node index $i \in [0, T-1]$, let layer(i) map it to its layer number $l \in [0, N_L - 1]$
- S_l be the set of indices in layer l

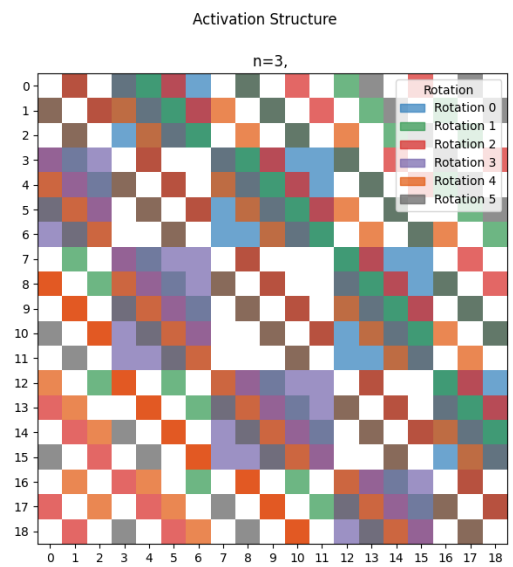

Figure 2: Overlay of the orientation-specific adjacency patterns for a HexNN with n=3. Each color corresponds to one directed orientation.

3.2.4 Directions as Rotations

As previously noted, properties such as layer sizes and layer weights for H_i will be the reverse for the geometric opposite network, $H_{(i+3) \bmod 6}$, and therefore, adjacency matrices, weight matrices, and other relevant matrices can all be considered symmetric (or at least will be in this paper). But this now introduces rotation as a potentially active participant in the training steps. Thus, for forward propagation, you can just iteratively compute:

$$x_{t+1} = R_k(f_d(W^T \times p_d(x_t)))$$

where x_t is the latent vector at step t, f_d is the hex net forward propagation through direction d, R_k is the rotation operator at k, the selected rotation step, and $p_d ismerelyapaddedmappingofx$

3.2.5 Adjacency Matrices

Then, the H_1, the only non-zero entries in A are defined as:

$$A_{ij} = \begin{cases} 1 & i \in S_l, j \in S_{l+1}, \text{ for some } l \in [0, N_L - 2] \\ 0 & \text{otherwise} \end{cases}$$

A symptom of this definition for H_1 is a block structure that gets created inside A^1. Each block $(l, l+1)$ is a dense matrix of ones with size $|S_l| \times |S_{l+1}|$ Thus, A^1 can be visualized as:

$$A^1 = \begin{bmatrix} 0 & B_0 & 0 & \dots & 0 \\ 0 & 0 & B_1 & \dots & 0 \\ \vdots & \vdots & \vdots & \ddots & \vdots \\ 0 & 0 & \dots & 0 & B_{N_L-2} \\ 0 & 0 & \dots & 0 & 0 \end{bmatrix}$$

where every $B_k \in \{0,1\}^{|S_k| \times |S_{k+1}|}$ is a matrix of all 1s. We can see this pattern hold as n grows for H_1 in 3

3.2.6 Example Training Step Flow with $n = 2$

In a network with $n = 2$,

- The layer sizes are $[2, 3, 2]$

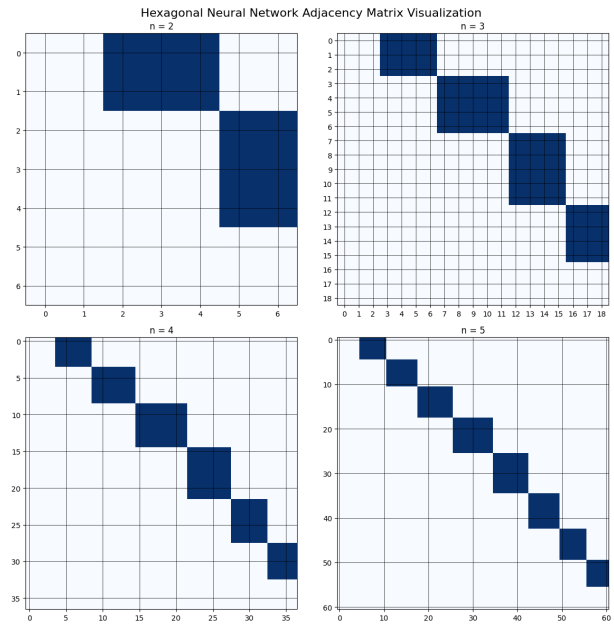

Figure 3: Adjacency Matrix showcasing the Block structure for H_1

- The total number of nodes is $T = 7$
- The Weight and Adjacency matrices $A, W \in \mathbb{R}^{7 \times 7}$
- The padded input vector $p_0(x)$ is

$$x_t = \begin{bmatrix} x_0 \\ x_1 \\ 0 \\ 0 \\ 0 \\ 0 \\ 0 \end{bmatrix}$$

- And the padded response vector is

$$\text{calculated } y_t = \begin{bmatrix} 0 \\ 0 \\ 0 \\ 0 \\ 0 \\ y_0 \\ y_1 \end{bmatrix}$$

If we interpret six co-dependent multi layer perceptrons as an ensemble with parameter tying, here are a few variants that we could try finding baselines within the guidance of ensemble based learning.

We will establish some baseline models, and their expected outputs. Then we will try five variants of these baselines that are based on exploits and manipulations that we can do exclusively because of hexagonal nature of the graph. To these five, we will then be testing some orthogonal knobs. We will establish what are the independent and dependent and testing variables that we will be fine tuning and sweeping through to test.

First, let's establish some baselines.

3.2.7 Baselines

A single model, without sharing, standard training Deep ensemble, six separate networks, no sharing, independent seeds. Snapshot ensemble / SWA: 1 network, multi snapshot averaging MC-Dropout: 1 net, inference-time dropout

3.2.8 Variants 1 - 5: Hex Ensemble Variants

Variant 1: Hard Rotation Sharing: One backbone tensor bank, each member 'applies a hex rotation' to filters adjacency before its forward. Only small member-specific heads (and optional tiny offsets) are unique. Combines group equivariance with ensembling

Variant 2: Soft sharing via low-rank delta (LoRA-style) Shared backbone θ; each member has $\Delta_i = A_i B_i$ (rank-r) on selected layers. Control r to trade diversity vs. parameters Optimal: permember BN/Layer Norm

Variant 3: Masked weight reuse Shared tensor bank W; each member uses a binary/ternary mask M_i (learned or fixed) such that $W \circ M_i$ Encourages orthogonal subspaces with negligible memory

Variant 4: Shared backbone, member heads only Classic multi head: 100 percent sahred feature extractor; six heads (Acts as lower bound on diversity)

Variant 5: Optimizer state-sharing vs isolatioin Same weights but either (i) one optimizer for all members (cotraining) or (ii) per member optimizers on their delta/heads only.

Variable	Options
Models	hex, mlp
Activations	linear, relu, leaky_relu
Losses	mean_squared_error, huber
Learning Rates	constant, exponential_decay
Dimension	Ladder up by twos, 2, 4, 6, 8

Dataset	Notes
identity	stuff
linear_scale	stuff
diagonal_scale	stuff
diagonal_linear	stuff
full_linear	stuff
low_rank_linear	stuff
affine	stuff
orthogonal_rotation	stuff

4 Data

5 Benchmark Families

The full cartesian product of all datasets and all known activation patterns, loss functions, learning rate, optimizers now with the new rotation operator would be thorough, it's not quite feasible for the scope of a paper, but it is considered for future work and exhaustive Hexnet dynamic surveying.

Therefore we define a Benchmark Family is a subset of this cartesian product where hold certain hyperparameter constants and test individual themes with thess operators. The theme of each family along with our findings are detailed below. The datasets are defined afterwards.

5.1 Family A - Linear Structure Recovery

Hyper parameters that we will sweep:

This helps us achieve parity with MLP with basic data sets and basic flow validation. We see hex geometry perform just as well as equivalent MLP ar-

chitecture, then it may be safe to assume isotropic maps are not trivially penalized.

We also see that on diagonal_scale, diagonal_linear, and low_rank_linear datasets, Hexnet performs better.

Hex Net loss and regression values fluctuate harshly after the rotation operator is applied, but this interference helps over all loss reach lower optimas on average.

When we specifically tested the orthogonal_rotation dataset, we noticed that it differed in performance to the other affine transformations encoded in datasets.

5.2 Base Case

Baselines for H_0 have been created where loss function, activation functions, and n are variable, and we can measure their performance with regression metrics such as scalar loss (according to the respective loss function), accuracy (root mean square deviation, as is standard for regression based data sets) and R^2 (the coefficient of determination).

5.3 Simulated Data Setup

I did things here (identity dataset, linear dataset, chat gpt outputted)

5.4 Real Data Sets

I did things with real data here

6 Results

This is what i got! For continual-learning context, see [1].

7 Conclusion

We introduced hexagonal neural networks as a topology-first variant of layered models and reported early training behavior on baseline tasks.

References

[1] James Kirkpatrick et al. "Overcoming catastrophic forgetting in neural networks". In: *Proceedings of the National Academy of Sciences* 114.13 (2017), pp. 3521–3526. DOI: 10.1073/pnas.1611835114.